

SORBONNE UNIVERSITÉ

Master 1 Informatique - STL

Rapport final du projet PSTL

Développement d'une interface graphique pour la visualisation de graphes en C

Auteurs

Chetouani Bilal - 21516679

Achabou Idris - 21500491

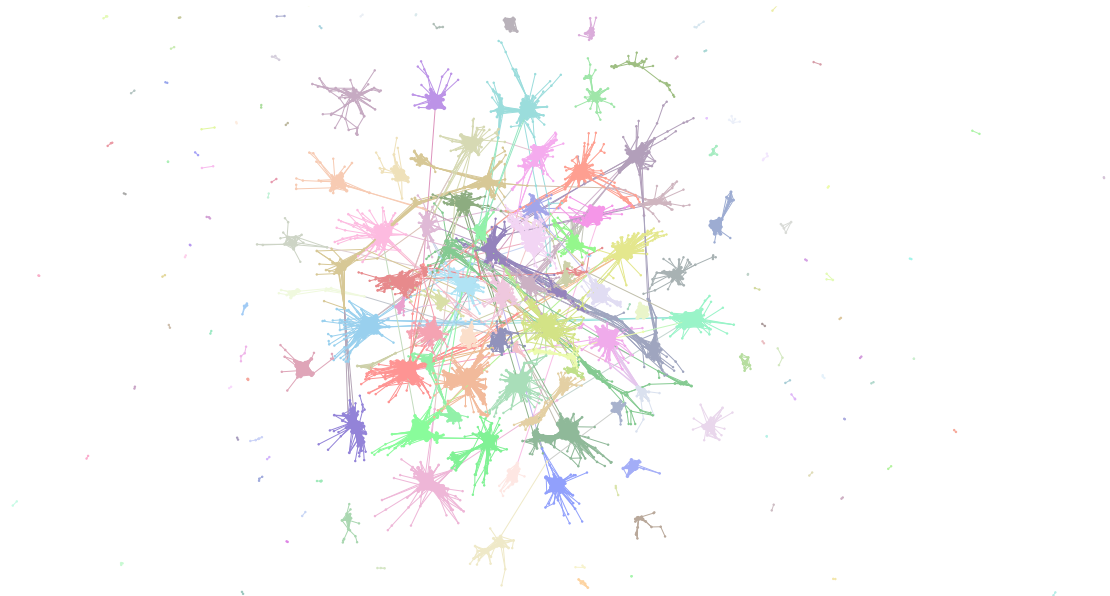
Encadrement

Olivier Bodini (Sorbonne Paris Nord)

Mehdi Naïma (ComplexNetworks, LIP6,
Sorbonne Université)

Dépôt Github

[https://github.com/
idris-ach2002/PSTL_25_
visualisation_graphes](https://github.com/idris-ach2002/PSTL_25_visualisation_graphes)



16 avril 2026

Table des matières

Glossaire	3
1 Introduction	5
1.1 Application	5
1.2 Contexte	6
1.3 Comparaison des deux versions	6
1.4 Déroulement	7
1.5 Étapes principales	8
1.6 Objectifs	8
2 Présentation technique	9
2.1 JavaFX et interface	9
2.2 OpenGL et rendu du graphe	9
2.3 JNI et moteur C	10
2.4 Algorithme de forces	10
2.4.1 Fonctionnement de l'update	10
2.4.2 Répulsion par clusters	11
2.4.3 Tentative d'optimisation par représentants	11
2.4.4 Rôle du k-means	12
3 Corrections et améliorations	14
3.1 Améliorations liées à Java	14
3.1.1 Refactorisation de l'architecture Java	14
3.1.2 Gestion des vues et intégration JOGL	14
3.1.3 Optimisations OpenGL côté Java	15
3.1.4 Essai de migration vers OpenGLFX	16
3.1.5 Réduction du couplage entre simulation et rendu	16
3.1.6 Refonte de l'affichage des données	16
3.1.7 Correction d'un problème d'affichage lors des changements d'onglet	17
3.1.8 Amélioration de l'interface utilisateur	17

3.1.9	Libération des ressources	17
3.2	Améliorations liées au C	17
3.2.1	Amélioration du chargement des données	17
3.2.2	Travail sur le calcul des forces	18
3.2.3	Utilisation d'une représentation CSR	18
3.2.4	Activation de flags de compilation adaptés	18
3.3	Améliorations hybrides C / Java	19
3.3.1	Compilation et portabilité	19
3.3.2	Optimisations du rendu	19
3.3.3	Résultats et limites sur grands graphes	20
3.3.4	Profilage des performances	20
4	Extensions du projet	22
4.1	Export PNG	22
4.2	Export SVG	22
4.3	Extensions liées à l'interface utilisateur	22
4.4	Prévisualisation : courbes de Bézier et épaisseur des arêtes	23
5	Critiques et extensions possibles	24
5.1	Évolutions possibles côté C	24
5.2	Évolutions possibles côté rendu	24
5.3	Évolutions possibles du calcul des forces	25
5.4	Extensions encore envisageables	25
6	Conclusion	26
	Annexes	27
.1	Version initiale	27
.2	Version finale	28

Glossaire

Terme	Définition simplifiée
Graphe	Structure composée de nœuds et d'arêtes, utilisée pour représenter des relations entre entités.
Nœud	Élément individuel du graphe, souvent associé à une personne, un objet, une donnée ou une observation.
Arête	Lien entre deux nœuds. Elle peut être orientée ou non, pondérée ou non.
Visualisation de graphes	Ensemble des techniques permettant d'afficher un graphe de manière lisible et exploitable pour l'analyse.
Layout	Méthode qui calcule la position des nœuds dans l'espace d'affichage.
Spatialisation	Calcul automatique de positions pour les nœuds d'un graphe dans le plan d'affichage.
ForceAtlas2	Algorithme de spatialisation de graphes reposant sur une métaphore de forces d'attraction et de répulsion [2].
Communauté	Groupe de nœuds plus fortement liés entre eux qu'avec le reste du graphe.
Clustering	Famille de méthodes qui regroupent des éléments selon une logique de proximité ou de similarité.
K-means	Algorithme de clustering qui répartit des points dans un nombre fixé de groupes à partir de centres calculés progressivement.
Similarité	Mesure numérique permettant d'évaluer à quel point deux lignes de données se ressemblent.
Seuil	Valeur à partir de laquelle une relation est retenue ou non lors de la construction du graphe.
Anti-arête	Lien utilisé dans certains modes pour représenter une forte dissimilarité entre deux nœuds.
JavaFX	Framework Java dédié aux interfaces graphiques riches [3, 4].
FXML	Langage déclaratif XML utilisé pour décrire des interfaces JavaFX [5].
JOGL	Binding Java donnant accès à l'API OpenGL [6].
OpenGLFX	Bibliothèque Java permettant d'intégrer une surface OpenGL dans une interface JavaFX [7].
LWJGL	Bibliothèque Java donnant accès à OpenGL et à d'autres API natives depuis la JVM [8].
OpenGL	API graphique bas niveau utilisée pour le rendu 2D et 3D accéléré [10, 11].

Terme	Définition simplifiée
Shader	Petit programme exécuté sur le GPU pour contrôler certains traitements de rendu.
JNI	Interface standard permettant à du code Java d'appeler des fonctions écrites en C ou dans d'autres langages natifs [9].
GPU	Processeur graphique, utilisé pour accélérer certains traitements d'affichage.
Buffer GPU	Zone mémoire côté carte graphique utilisée pour stocker des positions, couleurs ou indices de rendu.
CSR	Représentation compacte d'un graphe creux fondée sur des tableaux contigus, utilisée pour améliorer les parcours mémoire.
Pagination	Découpage d'un affichage ou d'un jeu de données en pages successives.
async-profiler	Profileur par échantillonnage pour applications Java, utilisé pour observer le coût CPU et produire des flame graphs [14].
Flame graph	Visualisation d'un profil d'exécution qui montre les piles d'appels et met en évidence les parties les plus coûteuses du programme [15].
DOT	Langage textuel utilisé pour décrire des graphes, notamment dans l'écosystème Graphviz [12].
PNG	Format d'image raster, adapté au stockage d'une image finale composée de pixels.
SVG	Format d'image vectoriel fondé sur une description géométrique des objets affichés [13].
Export raster	Production d'une image composée de pixels, par exemple au format PNG.
Export vectoriel	Production d'une image décrite géométriquement, par exemple au format SVG [13].
Courbe de Bézier	Courbe paramétrique définie par des points de contrôle, utilisée pour représenter des formes lisses ou des arêtes courbes.
Matrice de transformation	Matrice utilisée pour convertir les positions du graphe selon la caméra, le zoom ou le changement d'échelle avant affichage.
Coordonnées monde / écran	Distinction entre l'espace logique où le graphe est calculé et l'espace affiché dans la fenêtre.

1

Introduction

Ce rapport présente le travail réalisé dans le cadre du projet PSTL consacré à la visualisation interactive de graphes. Le projet s'appuie sur une application hybride combinant une interface JavaFX, un moteur de calcul en C et un rendu OpenGL. L'objectif général était de reprendre une base existante, de la rendre plus claire à utiliser et d'en faire une application plus simple à faire évoluer.

1.1 Application

L'application permet de visualiser un graphe à partir de deux types d'entrée : un fichier DOT ou un fichier CSV.

Dans le cas d'un fichier DOT, le graphe existe déjà. Le fichier décrit directement les nœuds, les arêtes et éventuellement certains attributs. Le travail de l'application consiste alors à lire cette structure, à calculer une disposition spatiale lisible et à l'afficher.

Dans le cas d'un fichier CSV, la situation est différente : le graphe n'existe pas encore, il doit être construit à partir des données. Chaque ligne du tableau est d'abord interprétée comme une entité du futur graphe. L'application compare ensuite les lignes deux à deux à l'aide d'une mesure de similarité choisie par l'utilisateur, par exemple la corrélation ou une distance.

À partir de ces comparaisons, l'application décide quelles relations doivent être conservées dans le graphe. Si deux lignes sont jugées suffisamment proches, une arête est créée entre les deux nœuds correspondants. À l'inverse, si elles sont jugées éloignées, l'application peut aussi créer une anti-arête, utilisée dans certains modes de calcul pour accentuer la séparation entre des éléments dissemblables. Entre ces deux cas, aucune relation n'est créée. Les seuils fixent ces limites : un seuil haut pour créer une arête, et un seuil bas pour créer une anti-arête.

Une fois le graphe construit, le moteur calcule la position des nœuds dans l'espace avec un algorithme de spatialisation inspiré des méthodes à forces. Les nœuds liés ont tendance à se rapprocher, tandis que la répulsion évite qu'ils se superposent. L'objectif est d'obtenir une disposition lisible de la structure du graphe.

L'application permet ensuite d'ajuster plusieurs paramètres visuels et algorithmiques, de lancer une détection de communautés, d'observer une prévisualisation puis d'exporter le résultat. Le projet relie donc plusieurs étapes : lecture des données, construction éventuelle du graphe, calcul des positions, rendu et export.

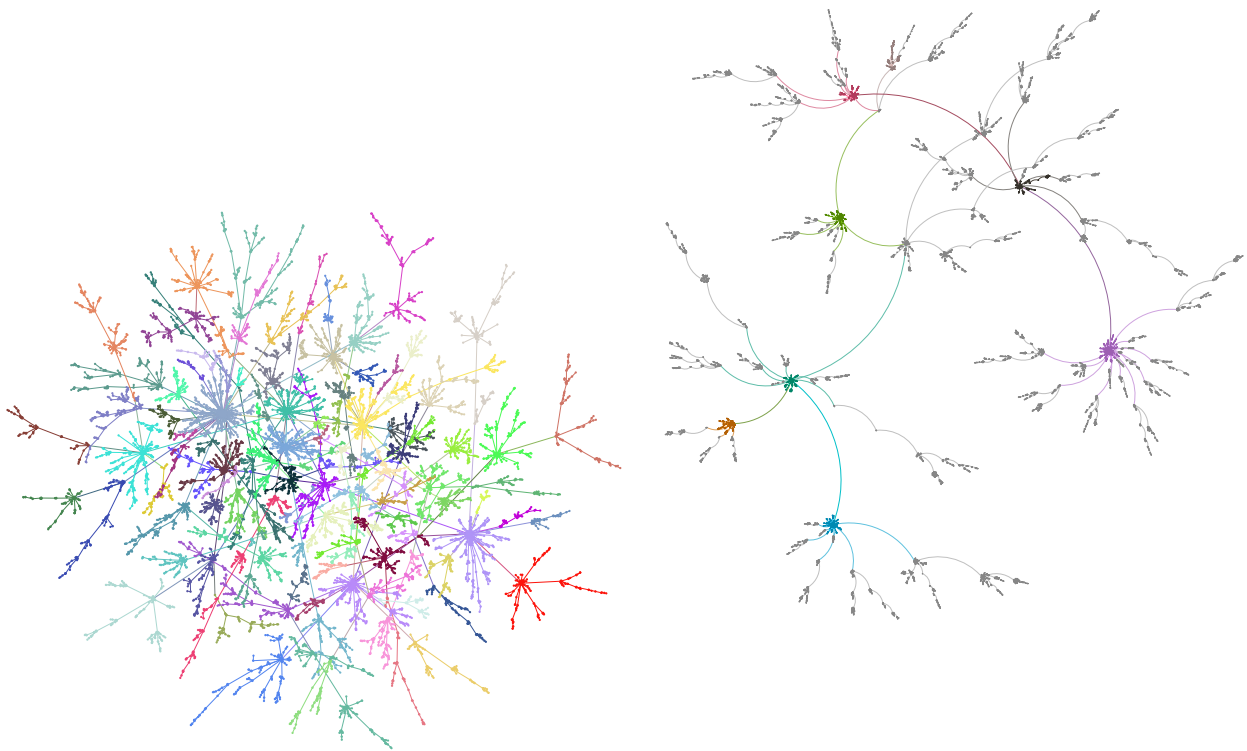


FIGURE 1.1 – Comparaison entre notre export vectoriel, à gauche, d’un graphe de 10 000 nœuds et arêtes et le rendu obtenu avec Gephi.

1.2 Contexte

Le code source dont nous avons hérité proposait déjà une interface graphique JavaFX, un moteur natif, un rendu OpenGL, une prise en charge des fichiers CSV et DOT, ainsi que du code C pour le calcul de similarité, le clustering et la détection de communautés.

Cette base restait cependant difficile à exploiter en pratique. Le lancement était fragile, notamment à cause de dépendances natives mal gérées, de chemins en dur et d’une compilation problématique sur macOS. La partie Java était concentrée dans quelques classes volumineuses, ce qui rendait l’ensemble difficile à lire et à faire évoluer. Plusieurs paramètres présents dans le moteur C n’étaient pas reliés à l’interface. Des bugs restaient visibles dans l’usage courant de l’application, par exemple lors du changement de vue, du rechargement de données ou de la fermeture de la fenêtre.

1.3 Comparaison des deux versions

La version initiale contenait déjà les principales briques du projet : interface JavaFX, rendu OpenGL avec JOGL, moteur C, support des fichiers CSV et DOT, ainsi que quelques algorithmes d’analyse. La version finale conserve cette base technique, mais la rend plus

structurée, plus lisible et plus simple à utiliser.

Les évolutions principales concernent l'organisation du code Java, le cycle de vie de l'application, l'interface utilisateur et la séparation entre rendu, modèle et moteur natif. Dans la version initiale, plusieurs responsabilités étaient mélangées dans les mêmes classes : interaction utilisateur, rendu, chargement natif, état de l'application et appels vers le moteur C. La version finale sépare davantage ces rôles, notamment avec des contrôleurs JavaFX spécialisés, des fichiers FXML, une couche de rendu dédiée et une interface plus claire vers le moteur natif.

La version finale reste fondée sur JOGL. Le rendu OpenGL est intégré dans JavaFX au moyen d'un `SwingNode` contenant un `GLJPanel`. Cette solution correspond à la version de référence du projet en fin de semestre : elle conserve la compatibilité avec l'existant, tout en bénéficiant des refactorisations et des corrections réalisées.

En parallèle, nous avons exploré des pistes d'optimisation plus ambitieuses sur la branche `test_optim`. Ces essais portent notamment sur le rendu OpenGL, la communication entre le rendu et la simulation, une tentative de migration vers OpenGLFX/LWJGL, et une approximation différente de la répulsion pour les grands graphes. Des captures d'écran illustrant la version initiale et la version finale sont présentées en annexe (Annexe 6).

1.4 Déroulement

Le travail sur le semestre s'est fait par étapes successives.

- **Fin janvier 2026** : prise en main du projet et lecture du code existant.
- **Début février 2026** : simplification du build et amélioration de la portabilité.
- **Février 2026** : refactorisation de la partie Java et clarification de l'architecture.
- **Février – mars 2026** : correction des bugs principaux de l'interface et du cycle de vie de l'application.
- **Mars 2026** : amélioration de l'ergonomie et clarification des paramètres utilisateur.
- **Mars 2026** : reprise de plusieurs parties du moteur C.
- **Mars – avril 2026** : travail sur le rendu OpenGL, la prévisualisation et les performances.
- **Avril 2026** : ajout des exports PNG et SVG et finalisation des fonctionnalités principales.
- **Avril – mai 2026** : rédaction du rapport et exploration d'optimisations sur la branche `test_optim`.

Cette progression montre que le projet n'a pas consisté à ajouter directement des fonctionnalités. Il a d'abord fallu rendre l'existant compréhensible, compilable et modifiable, avant de pouvoir

enrichir l'application.

1.5 Étapes principales

Le fonctionnement global peut se résumer ainsi :

1. charger un fichier DOT ou CSV ;
2. construire un graphe si nécessaire à partir des données tabulaires ;
3. calculer une disposition spatiale des nœuds ;
4. appliquer éventuellement des regroupements ou une détection de communautés ;
5. visualiser, ajuster les paramètres et exporter le résultat.

Ces étapes structurent l'ensemble du projet et expliquent la coexistence de plusieurs couches techniques : interface JavaFX, rendu OpenGL, moteur natif et traitements de données.

1.6 Objectifs

Lors des réunions de suivi avec nos encadrants (M. Olivier Bodini et M. Mehdi Naïma), les priorités suivantes ont été définies :

1. **Portabilité** : faire fonctionner l'application sur Linux, macOS et Windows, avec un système de compilation plus clair.
2. **Refactorisation** : réorganiser le code, surtout Java, pour mieux séparer les responsabilités et faciliter l'évolution future.
3. **Complétude fonctionnelle** : exposer les paramètres utiles du moteur et ajouter des fonctionnalités d'export et de prévisualisation.
4. **Stabilisation** : corriger les bugs visibles et rendre le comportement général de l'application plus cohérent.
5. **Performances** : identifier les limites du rendu, des transferts Java/C et du calcul natif, puis tester des pistes d'amélioration.
6. **Documentation** : fournir un README plus clair et produire un rapport explicitant les choix effectués.

Ce rapport décrit les principales améliorations réalisées dans ce cadre, en détaillant à la fois les corrections apportées au code existant et les choix d'architecture qui ont permis de faire évoluer l'application.

2

Présentation technique

Cette partie présente brièvement les principales briques techniques du projet. L'objectif n'est pas de détailler toute l'implémentation, mais d'expliquer comment JavaFX, OpenGL, JNI et le calcul de forces interviennent dans l'application.

2.1 JavaFX et interface

JavaFX est utilisé pour construire l'interface graphique. Une application JavaFX repose sur une fenêtre principale, appelée **Stage**, contenant une **Scene**. Cette scène contient un arbre de composants graphiques, appelé **Scene Graph**. Les boutons, panneaux, zones de texte et conteneurs sont donc organisés dans une hiérarchie.

Dans notre projet, l'interface est décrite avec des fichiers FXML. Le FXML permet de séparer la structure visuelle de la logique Java. Chaque fichier FXML est associé à un contrôleur, qui récupère les composants avec `@FXML` et réagit aux actions utilisateur.

JavaFX impose aussi une contrainte importante : les modifications de l'interface doivent se faire sur le **JavaFX Application Thread**. Si le moteur C ou un autre thread produit un résultat, la mise à jour graphique doit être renvoyée vers ce thread. Une mauvaise gestion de cette séparation peut provoquer de la latence ou des comportements incohérents dans l'interface.

2.2 OpenGL et rendu du graphe

OpenGL est utilisé pour dessiner le graphe de manière accélérée. L'idée est d'éviter de représenter chaque nœud ou chaque arête comme un composant JavaFX indépendant. Une telle approche serait coûteuse sur des graphes volumineux.

Dans la version finale, le rendu repose sur JOGL. Le composant principal est un `GLJPanel`, intégré dans l'interface JavaFX au moyen d'un `SwingNode`. Le `render` implémente `GLEventListener`, ce qui permet à JOGL d'appeler les méthodes d'initialisation, de redimensionnement, d'affichage et de libération du contexte OpenGL.

Le rendu repose sur des buffers envoyés au GPU. Ces buffers contiennent les informations utiles à l'affichage, par exemple les positions des sommets, les indices des arêtes, les couleurs ou les tailles. OpenGL utilise ensuite ces données pour dessiner les nœuds et les arêtes.

Le rendu suit globalement les étapes suivantes :

1. récupérer les positions calculées par le moteur ;
2. préparer les données nécessaires au rendu ;
3. mettre à jour les buffers GPU ;
4. appliquer la caméra, le zoom et la projection ;
5. dessiner les arêtes puis les sommets.

Cette organisation limite le nombre d'objets Java manipulés à chaque image. Les essais de la branche `test_optim` ont également permis d'étudier une intégration plus directe via OpenGLFX/LWJGL, afin d'évaluer les gains possibles côté rendu.

2.3 JNI et moteur C

Le moteur de calcul est écrit en C pour traiter les opérations coûteuses plus efficacement que si elles étaient directement réalisées côté interface. Java appelle ce moteur grâce à JNI, l'interface qui permet à la JVM d'exécuter du code natif.

Le rôle de JNI est de faire le pont entre deux environnements différents. Côté Java, la mémoire est gérée automatiquement par la JVM. Côté C, la mémoire doit être gérée explicitement. Cette séparation impose de faire attention aux allocations, aux libérations et aux transferts de données.

Dans le projet, JNI est utilisé pour lancer les traitements natifs et récupérer les données utiles au rendu. Les buffers directs permettent de limiter certaines copies entre Java et C, ce qui est important lorsque les positions d'un grand nombre de nœuds doivent être mises à jour régulièrement.

2.4 Algorithme de forces

La spatialisation du graphe repose sur une logique de forces. Les arêtes attirent les nœuds reliés, tandis que la répulsion évite que tous les nœuds se placent au même endroit. L'objectif n'est pas de calculer une position unique ou optimale, mais d'obtenir une disposition assez lisible pour comprendre la structure du graphe.

2.4.1 Fonctionnement de l'update

La fonction centrale du moteur est `updatePositions`. À chaque tick, elle exécute les étapes suivantes :

1. recalculer plusieurs seuils à partir de la taille courante de la fenêtre ;
2. appliquer les forces liées aux arêtes normales ;

3. appliquer la répulsion principale entre nœuds ;
4. appliquer les forces liées aux anti-arêtes ;
5. intégrer les forces dans les positions ;
6. mettre à jour les clusters ;
7. copier les positions dans le buffer direct utilisé par Java.

On peut donc décomposer le coût d’une mise à jour en plusieurs parties : le parcours des arêtes normales, le parcours des anti-arêtes, la répulsion entre nœuds, l’intégration des positions et la mise à jour des clusters.

2.4.2 Répulsion par clusters

La répulsion n’est pas calculée entre toutes les paires de nœuds du graphe. Elle est calculée à l’intérieur des clusters. Pour un cluster contenant s_c nœuds, le code parcourt les couples de nœuds du cluster. Le coût de cette partie est donc proportionnel à s_c^2 .

Sur l’ensemble des clusters, le coût de la répulsion exacte est de l’ordre de :

$$O\left(\sum_{c=1}^K s_c^2\right)$$

où K désigne le nombre de clusters et s_c la taille du cluster c .

Le coût dépend surtout de la manière dont les nœuds sont répartis entre les clusters. Si les clusters sont équilibrés, le coût est plus contenu. Si un cluster devient très gros, cette partie devient plus coûteuse.

Le coût complet d’une mise à jour, dans le cas exact, peut être résumé ainsi :

$$O\left(E + A + N + \sum_{c=1}^K s_c^2\right)$$

où E est le nombre d’arêtes normales, A le nombre d’anti-arêtes et N le nombre de nœuds.

2.4.3 Tentative d’optimisation par représentants

La tentative d’optimisation présente dans la branche `test_optim` modifie le calcul de la répulsion principale pour les grands graphes. Cette branche ne correspond pas à la version finale du `main` : elle constitue une expérimentation séparée visant à étudier le passage à l’échelle.

Dans la version principale, la répulsion est calculée de manière exacte à l’intérieur des clusters. La branche `test_optim` conserve cette répulsion exacte pour les graphes de taille modérée. Dans le code, la bascule vers l’approximation ne se fait que lorsque le nombre de nœuds

vivants atteint environ 60 000. En dessous de ce seuil, la répulsion reste donc une répulsion paire à paire à l'intérieur des clusters courants.

Cependant, cette conservation doit être nuancée. Même lorsque la répulsion exacte est encore utilisée, la branche `test_optim` ne reproduit pas exactement le comportement de la version principale. Plusieurs éléments ont été modifiés en même temps : la manière de mettre à jour les clusters, le traitement du k-means, la géométrie utilisée dans les forces et la gestion du recentrage du graphe. Cela explique que les rendus puissent être très différents entre la version principale et la tentative d'optimisation, même sur des graphes qui restent sous le seuil d'activation des représentants.

Pour les grands graphes, la branche `test_optim` construit un petit ensemble de représentants, appelés *landmarks* dans le code. Ces représentants sont des nœuds réels du graphe, sélectionnés par hachage. Le but était d'éviter une grille spatiale, car les anciennes approches par grille pouvaient imposer des formes rectangulaires visibles dans le rendu.

Le nombre de représentants reste borné. Dans le code, il dépend de la racine carrée du nombre de nœuds actifs, puis il est limité entre 32 et 80. Chaque représentant porte un poids qui approxime une partie des nœuds non sélectionnés.

Au lieu de calculer toutes les interactions intra-cluster, la version expérimentale calcule alors une répulsion entre chaque nœud et ces représentants. Si R est le nombre de représentants, le coût de cette partie devient :

$$O(N \times R)$$

Comme R reste borné, le coût devient beaucoup plus contrôlable sur les grands graphes. Cette approximation améliore le coût de calcul, mais elle change aussi la dynamique de spatialisation. Elle doit donc être considérée comme une piste expérimentale, et non comme une optimisation strictement équivalente à la version principale.

2.4.4 Rôle du k-means

Le moteur contient aussi un mode k-means. Lorsqu'il est activé, une itération de k-means est exécutée périodiquement plutôt qu'une boucle complète jusqu'à convergence. Ce choix évite de rendre `updatePositions` trop variable en temps d'exécution, car une convergence complète pourrait prendre un temps différent selon le graphe.

Le k-means améliore nettement le résultat dans les deux approches. Il produit des groupes plus cohérents et aide à réduire certains effets artificiels. La qualité finale de l'algorithme dépend encore du nombre de clusters, de l'évolution des centres et de l'équilibre entre attraction et répulsion.

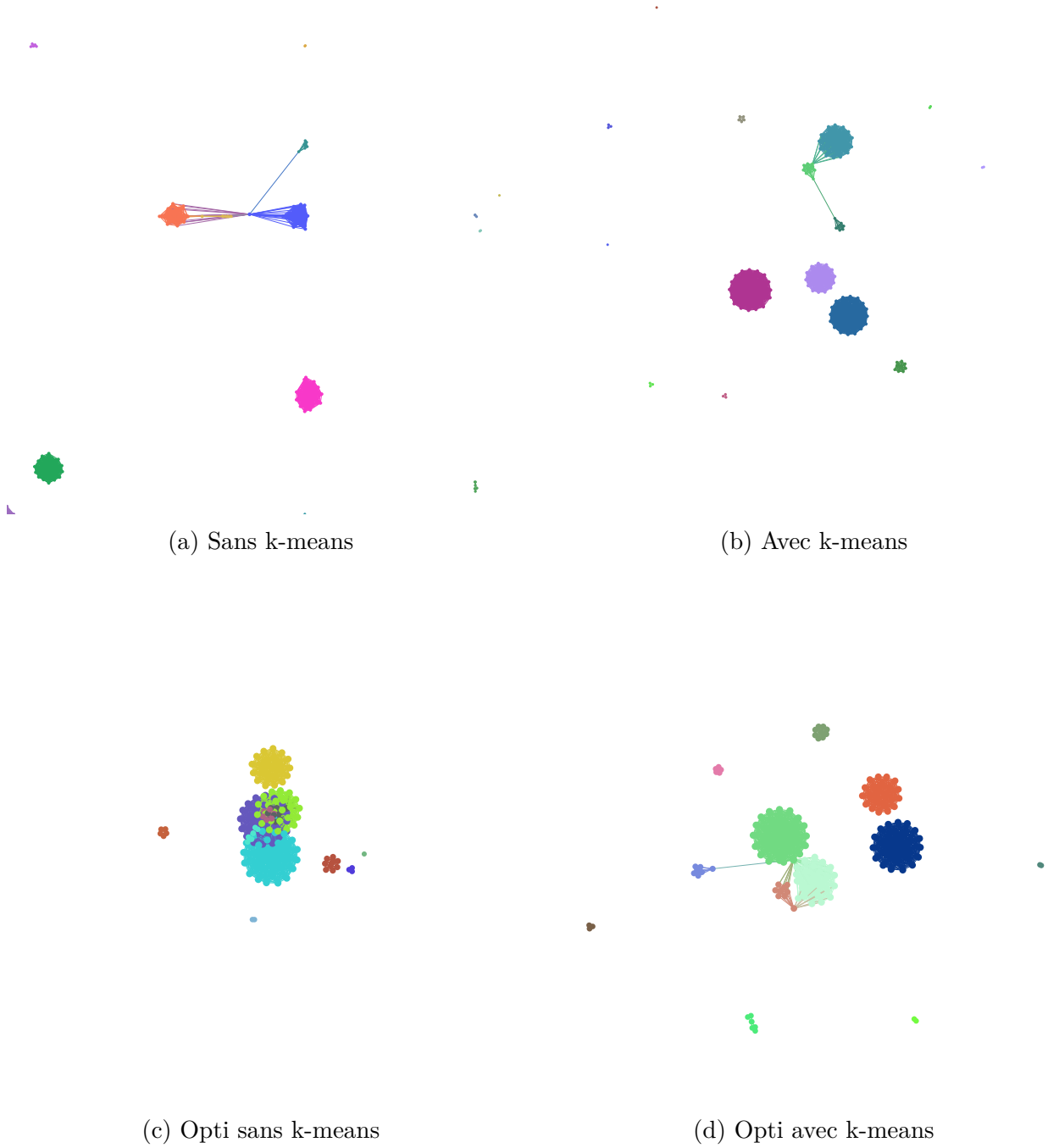


FIGURE 2.1 – Comparaison des résultats

3

Corrections et améliorations

3.1 Améliorations liées à Java

3.1.1 Refactorisation de l'architecture Java

La partie Java a fait l'objet d'une refactorisation importante par rapport à la version initiale. Dans l'ancien code, une grande partie des responsabilités était concentrée dans quelques classes chargées, en particulier **Graph** et **GraphVue**. Ces classes mélangeaient la logique de l'interface, le rendu OpenGL, les interactions utilisateur, le chargement de la bibliothèque native et une partie de l'orchestration générale.

Dans la nouvelle version, les responsabilités sont réparties dans des packages plus clairs. L'objectif était surtout de rendre le projet plus lisible et de faciliter les modifications futures.

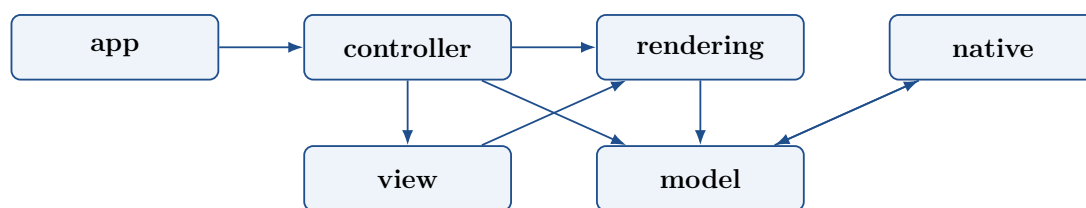


FIGURE 3.1 – Organisation simplifiée des principaux packages Java.

Le package **app** contient le point d'entrée de l'application. Le package **controller** regroupe les contrôleurs JavaFX. Le package **view** contient les éléments liés à l'affichage JavaFX et aux interactions utilisateur. Le package **rendering** isole le rendu OpenGL. Le package **model** représente les données du graphe. Le package **native** centralise le lien avec le moteur C.

Cette séparation rend le code plus lisible et facilite la localisation des modifications. Un changement d'interface, de modèle ou de rendu peut être raisonné dans une zone plus précise du projet.

3.1.2 Gestion des vues et intégration JOGL

Un autre point important du refactor Java a concerné l'intégration du rendu JOGL dans l'interface. Lors des premiers essais, nous avons utilisé un `StackPane` pour gérer le changement entre les différentes vues. Cette solution paraissait simple, mais elle a introduit un problème difficile à diagnostiquer : les panneaux principal et d'aperçu n'avaient pas la même taille, et le `StackPane` avait tendance à propager une taille maximale commune. En pratique, cela faussait

les dimensions réellement utilisées par le panneau principal, ce qui entraînait un décalage dans les coordonnées écran vers graphe. Ce défaut se répercutait directement sur les interactions, en particulier sur la sélection des nœuds, qui devenait erronée. Nous avons finalement remplacé cette logique par une organisation mieux adaptée, où chaque vue conserve son propre espace et ses propres dimensions grâce à un `TabPane`. Ce changement a permis de retrouver une cohérence entre la taille affichée, les calculs de projection et les interactions utilisateur.

3.1.3 Optimisations OpenGL côté Java

La partie Java liée au rendu OpenGL a demandé un travail précis, car les problèmes rencontrés n'étaient pas toujours signalés par des exceptions ou des logs explicites. Souvent, un défaut se manifestait uniquement par un problème visuel : arêtes qui clignotent, sommets mal positionnés, couleurs incohérentes, rendu partiel ou frame figée. Nous avons donc adopté une approche pragmatique : réduire le nombre d'éléments dynamiques dans la boucle de rendu, séparer la préparation des données et leur affichage, puis rendre chaque étape du pipeline plus lisible.

Dans l'ancien code, le rendu, l'interaction, la gestion du graphe et l'animation JOGL étaient davantage imbriqués. Dans la nouvelle version, nous avons isolé le pipeline OpenGL. La classe `GraphRenderer` se concentre sur l'orchestration du rendu, tandis que la préparation des données GPU est prise en charge par des classes dédiées comme `VertexGpuBuffer` et `EdgeGpuBuffer`. Cette séparation permet de mieux localiser l'origine d'un problème visuel : données CPU, transfert GPU, état OpenGL ou shader.

Une optimisation a consisté à préparer les données dans des tableaux réutilisables, puis à envoyer ces blocs vers le GPU avec des buffers dédiés, au lieu de reconstruire continuellement des objets ou des structures intermédiaires dans la boucle d'affichage. Pour les sommets, nous avons séparé les positions, tailles, couleurs et états de visibilité. Pour les arêtes, nous avons distingué le cas des segments droits et celui des courbes, avec une génération contrôlée des points nécessaires avant l'envoi au GPU. Cette logique évite de disperser les calculs dans plusieurs endroits du code et permet de mieux maîtriser le coût de chaque frame.

Nous avons aussi porté une attention particulière aux allocations mémoire dans le rendu. L'idée générale a été d'éviter autant que possible les créations d'objets dans les chemins exécutés à chaque frame. Même lorsque ces allocations ne provoquent pas d'erreur visible, elles peuvent introduire des ralentissements difficiles à diagnostiquer. Nous avons donc privilégié des buffers redimensionnables mais réutilisés, avec une politique d'agrandissement progressive, plutôt que de réallouer sans cesse.

Le même raisonnement a guidé le travail sur les shaders et sur l'état OpenGL. Nous avons cherché à centraliser leur création, à limiter les changements d'état inutiles et à rendre le parcours du pipeline plus prévisible. En pratique, lorsqu'un bug graphique est silencieux, il faut rendre le chemin de rendu le plus simple possible, limiter les effets de bord, isoler les responsabilités et supprimer ce qui varie inutilement d'une frame à l'autre.

Ce travail a amélioré la lisibilité du code de rendu et a facilité le diagnostic des problèmes graphiques. Il a aussi servi de base aux essais de la branche `test_optim`, où nous avons

mesuré plus précisément le coût du rendu avec `async-profiler`.

3.1.4 Essai de migration vers OpenGLFX

Nous avons étudié une migration du rendu JOGL vers OpenGLFX/LWJGL. L'objectif était de réduire les difficultés liées à l'intégration du rendu dans JavaFX et d'éviter le passage par `SwingNode` et `GLJPanel`.

Cette piste a montré un gain potentiel pendant les essais. Elle pouvait réduire certains coûts côté interface et rendre le pipeline graphique plus direct. Cependant, elle demandait de modifier beaucoup d'éléments en même temps : création du contexte OpenGL, intégration JavaFX, gestion des événements utilisateur, buffers GPU, export PNG et rendu des arêtes.

Cette migration a donc été conservée comme piste d'amélioration sur une branche séparée. La version principale garde le rendu JOGL, qui correspond à l'architecture finalisée dans le main.

3.1.5 Réduction du couplage entre simulation et rendu

Nous avons aussi travaillé sur le couplage entre la simulation C et le rendu Java. Dans ce type d'application, il faut éviter que le renderer bloque trop directement sur le moteur natif, ou que l'interface tente d'afficher trop d'états intermédiaires.

L'idée testée était de faire en sorte que le rendu utilise l'état disponible le plus récent, plutôt que de forcer l'affichage de toutes les étapes produites par la simulation. Cela correspond mieux à une application interactive : si la simulation avance plus vite que l'écran ne peut afficher, il est préférable de montrer le dernier état disponible.

Cette logique a surtout été approfondie dans les essais de la branche `test_optim`. Elle complète le travail réalisé sur les buffers et sur l'organisation du rendu.

3.1.6 Refonte de l'affichage des données

La version initiale présentait une limite dans le panneau de données : toutes les lignes étaient chargées et affichées d'un seul coup dans la même vue. Cela revenait à créer immédiatement un grand nombre de nœuds JavaFX dans le conteneur d'affichage. En pratique, une partie des données cessait d'apparaître correctement.

Nous avons remplacé cette logique par une pagination, de sorte que seules les données de la page courante soient instanciées et affichées. Cette modification a rendu le panneau de données utilisable sur des fichiers volumineux.

3.1.7 Correction d'un problème d'affichage lors des changements d'onglet

Nous avons également dû corriger un bug lié au changement d'onglet dans l'interface JavaFX. Dans certaines situations, la fenêtre ou le panneau OpenGL restait affiché ou continuait à se comporter comme s'il était encore actif, alors que l'utilisateur avait déjà changé de contexte.

Ce type de défaut était particulièrement gênant car il ne produisait pas forcément de message d'erreur clair, mais se manifestait par un affichage incohérent, un rendu figé ou des interactions qui semblaient encore attachées à une ancienne vue.

Le correctif a consisté à mieux contrôler le cycle de vie des panneaux graphiques : attachement explicite lors de l'affichage, détachement lors du changement de vue, et mise à jour plus propre des composants qui dépendent du contexte courant. Cela a permis d'éviter qu'une ancienne vue graphique continue à interférer avec la nouvelle.

3.1.8 Amélioration de l'interface utilisateur

Nous avons revu l'interface utilisateur pour la rendre plus directe et plus compréhensible. L'utilisateur accède plus rapidement à l'espace de travail, avec une logique de configuration recentrée sur les paramètres utiles. Nous avons aussi ajouté des tooltips, amélioré les paramètres dynamiques et distingué plus clairement les réglages de lancement des réglages modifiables pendant l'exécution.

3.1.9 Libération des ressources

Nous avons traité un problème de terminaison qui existait dans la version initiale. Même après la fermeture de la fenêtre, le programme pouvait continuer à tourner en arrière-plan, ce qui montrait que certaines ressources n'étaient pas correctement libérées. Nous avons donc revu la séquence de fermeture afin d'arrêter explicitement la boucle de rendu, détacher les panneaux graphiques, retirer les écouteurs encore actifs et libérer les ressources dans le bon ordre.

3.2 Améliorations liées au C

3.2.1 Amélioration du chargement des données

Le chargement des données a été un point important du travail sur la partie C. Dans la version initiale, certains graphes volumineux étaient difficiles à charger, soit parce que les structures mémoire étaient mal réinitialisées lors d'un rechargement, soit parce que la construction du graphe passait par trop d'objets intermédiaires. Cela limitait en pratique la taille des graphes exploitables.

Nous avons corrigé la séquence de chargement afin de libérer proprement les anciennes structures, de reconstruire les buffers natifs de manière plus compacte et de réduire les conversions inutiles entre C et Java. Ce travail a amélioré les rechargements successifs et a limité certaines erreurs mémoire.

3.2.2 Travail sur le calcul des forces

Le calcul des forces a été l'un des points les plus importants du projet. Il influence à la fois le temps d'exécution et la qualité visuelle obtenue après spatialisation.

Le code final utilise une répulsion exacte par clusters pour les graphes modestes. Cette organisation évite de comparer tous les nœuds du graphe entre eux, tout en conservant une répulsion détaillée à l'intérieur des groupes. Les performances dépendent alors de la répartition des nœuds entre clusters.

La tentative d'optimisation de la branche `test_optim` utilise une autre approche pour les grands graphes. Elle sélectionne un petit nombre de représentants réels du graphe, choisis par hachage, puis applique une répulsion entre chaque nœud et ces représentants. Le nombre de représentants reste borné entre 32 et 80. Cette stratégie vise à réduire le coût de la répulsion tout en évitant les effets de grille qui donnaient des formes rectangulaires visibles dans certaines versions précédentes.

Cette tentative améliore le temps de calcul dans certains cas et ouvre une piste intéressante pour la suite. Le point à poursuivre concerne surtout la qualité de la séparation visuelle des communautés : la réduction du coût doit rester compatible avec une disposition claire.

Le k-means améliore clairement les deux approches. Il aide à obtenir des groupes plus cohérents et à limiter certains effets artificiels. La qualité finale dépend cependant encore de l'équilibre entre attraction, répulsion, nombre de clusters et mise à jour des centres.

Ce travail montre que la spatialisation ne se résume pas à accélérer une formule. Une approximation plus rapide doit rester compatible avec une image lisible, tandis qu'un calcul plus précis peut devenir coûteux. Le point central est donc le compromis entre coût de calcul, stabilité et séparation visuelle des communautés.

3.2.3 Utilisation d'une représentation CSR

Dans la partie C, nous avons aussi revu la représentation du graphe utilisée par les algorithmes de communautés. La représentation CSR améliore la localité mémoire et réduit les accès indirects. Elle est utile pour Louvain et Leiden, qui parcourent souvent les voisins des nœuds.

3.2.4 Activation de flags de compilation adaptés

Nous avons également revu la compilation native. L'ajout de `-std=c11` force un standard explicite, ce qui améliore la portabilité. Les flags `-Wall` et `-Wextra` aident à détecter plus tôt certaines erreurs discrètes. Les flags d'optimisation comme `-O3` améliorent les parties

calculatoires du moteur.

L'utilisation de `-march=native` permet d'exploiter les instructions disponibles sur la machine de compilation. Ce choix améliore les performances localement, mais peut produire un binaire moins universel. Pour une distribution plus large, il serait préférable de prévoir deux modes : un mode générique portable et un mode optimisé pour la machine locale.

3.3 Améliorations hybrides C / Java

3.3.1 Compilation et portabilité

La compilation sur macOS a mis en évidence plusieurs faiblesses de l'organisation initiale du projet. Les chemins vers Java, JavaFX ou les bibliothèques natives étaient parfois écrits en dur. Cela rendait le lancement dépendant de l'environnement exact de l'utilisateur.

Nous avons recentré la partie Java autour d'un module Maven plus simple et isolé la compilation native dans `graph-native`. Le chargement de la bibliothèque native a aussi été centralisé dans `GraphNativeEngine`. La version finale tient compte des différences entre Linux, macOS et Windows pour les extensions natives.

3.3.2 Optimisations du rendu

Nous avons rapidement constaté que la version d'origine n'était pas adaptée au chargement de graphes de grande taille, alors même que c'était l'un des objectifs importants du projet. Le travail d'optimisation a donc porté à la fois sur la partie Java, sur le rendu OpenGL et sur la partie native en C.

Un premier point important a concerné le transfert des positions entre le moteur natif et l'interface Java. Au lieu de récupérer les positions par appels répétés provoquant des allocations inutiles, nous avons mis en place un buffer partagé, alloué une seule fois, puis mis à jour directement côté C et lu côté Java. Cela a permis de réduire le coût des échanges entre les deux couches.

Nous avons également fait le choix de simplifier la concurrence côté natif. Plutôt que d'introduire de nombreux verrous dans le code C, avec les risques classiques de blocage, d'oubli de libération ou de sections critiques trop larges, nous avons préféré limiter les accès sensibles en imposant un seul appel JNI à la fois pour les opérations critiques. Ce compromis rend le code plus lisible, tout en restant acceptable en performance, car la partie réellement coûteuse concerne surtout la mise à jour des positions.

Côté Java, nous avons davantage exploité les mécanismes concurrents du langage, en particulier les collections concurrentes, afin de simplifier la gestion des données partagées entre le calcul et le rendu. Dans `GraphModel`, les sommets et les arêtes sont stockés dans des `ConcurrentLinkedQueue`, tandis que l'accès aux sommets par identifiant repose sur une `ConcurrentHashMap`. Les compteurs visibles utilisent des `AtomicInteger`, et plusieurs paramètres lus par le rendu sont déclarés en `volatile`.

Cette organisation permet de suivre un modèle proche du producteur-consommateur : une partie du programme met à jour les données, tandis qu'une autre les lit pour l'affichage. Les lectures fréquentes du rendu ne nécessitent donc pas un verrou manuel autour de chaque parcours. Les modifications structurelles plus sensibles restent regroupées derrière un verrou d'écriture Java avec `synchronized (writeLock)`, notamment lors de la reconstruction du graphe, de l'ajout de sommets ou d'arêtes et de l'application des filtres.

Pour la caméra et le panning, nous avons aussi travaillé sur une logique de double buffering. L'idée consiste à écrire les nouvelles valeurs dans une zone dédiée, tandis que le rendu lit dans une autre. Une fois la mise à jour terminée, les références peuvent être échangées proprement. Cela limite les accès concurrents directs sur les mêmes données sans ajouter de synchronisation lourde dans le chemin courant.

Enfin, nous avons réduit les allocations dans la boucle de rendu OpenGL, car ce type de coût devient pénalisant sur des graphes volumineux. Le même effort d'optimisation a aussi été appliqué à certains traitements CSV, en particulier autour du calcul des seuils, afin de limiter les surcoûts inutiles lors du prétraitement.

3.3.3 Résultats et limites sur grands graphes

Les tests sur grands graphes ont montré que la version principale du projet constitue un progrès net par rapport à la base initiale : l'application est plus structurée, les étapes sont plus claires, les exports sont disponibles, et l'intégration entre interface, rendu et moteur natif est plus lisible. La branche principale est capable d'atteindre des graphes de taille 100 000 nœuds et 100 000 arêtes ce qui reste inférieur aux performances de Gephi.

Le profilage a montré que le coût se répartit entre plusieurs couches. Une partie vient du moteur C, en particulier du calcul des forces. Une autre partie vient de l'interface, du rendu JOGL et des transferts entre les différentes couches.

Une branche séparée, nommée `test_optim`, a été créée pour explorer des optimisations plus lourdes. Ces essais ont permis d'atteindre des graphes d'environ 500 000 nœuds et 499 000 arêtes dans certaines conditions ce qui pouvait rivaliser voir dépasser les performances de Gephi. Cette branche montre la marge d'amélioration possible lorsque l'on agit à la fois sur le rendu et sur le calcul.

3.3.4 Profilage des performances

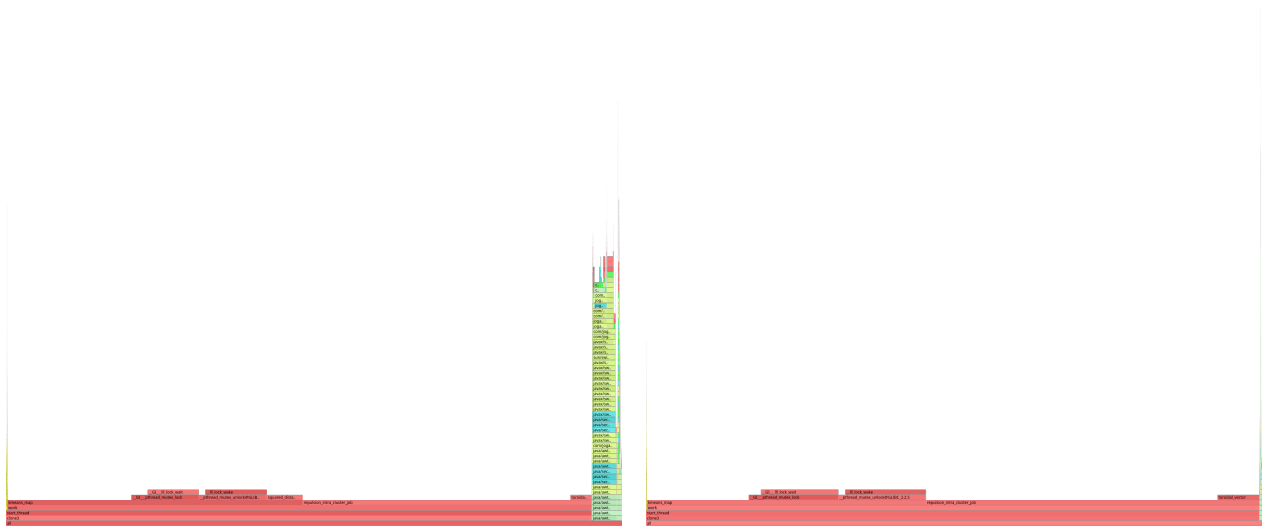
Les problèmes de performance n'ont pas été traités uniquement de manière intuitive. Nous avons utilisé `async-profiler` afin d'observer où l'application passait réellement du temps pendant l'exécution. Cet outil permet de profiler une application Java en cours d'exécution et de produire des flame graphs, qui visualisent les piles d'appels ainsi que le poids relatif des différentes parties du programme.

Ces mesures ont été réalisées sur un même graphe de 100 000 nœuds et 100 000 arêtes. Le premier flame graph, obtenu sur la version finale, montre que le coût principal vient du calcul

de l'algorithme, tandis que le rendu reste visible mais moins dominant. Le second flame graph, obtenu après des optimisations centrées sur le rendu, montre une réduction du coût de la partie graphique. Comme l'algorithme n'a pas été modifié dans cet essai, il reste naturellement la partie la plus coûteuse de l'exécution.

Ces mesures confirment donc que l'optimisation du rendu apporte un gain réel, mais que le passage à l'échelle dépend fortement du calcul de spatialisation qu'il faudra donc optimiser dans le futur.

Dans nos captures, les appels C apparaissent en rouge, tandis que les appels Java apparaissent avec d'autres couleurs. Plus une barre est large, plus le coût CPU correspondant est important.



(a) Flame graph de la version finale

(b) Flame graph après optimisations du rendu dans la version de test

FIGURE 3.2 – Comparaison de flame graphs obtenus avec `async-profiler` sur un même graphe d'environ 100 000 nœuds et 100 000 arêtes. À gauche, la version finale montre que le calcul de l'algorithme domine le coût total. À droite, après optimisation du rendu sans modification de l'algorithme, le coût graphique diminue, et l'algorithme devient encore plus clairement le point principal à traiter pour la suite.

4

Extensions du projet

4.1 Export PNG

Nous avons ajouté un export au format PNG afin de produire une image directement exploitable du graphe. Cet export repose sur une logique de rasterisation : la scène est transformée en une image composée de pixels. Concrètement, il ne s'agit pas d'exporter une structure abstraite du graphe, mais son apparence finale telle qu'elle est affichée.

L'export PNG a demandé de faire attention au contexte graphique actif. Un export OpenGL doit être réalisé au bon moment du cycle de rendu afin de récupérer correctement le contenu de la surface. Nous avons donc adapté cette partie pour éviter d'exporter une image vide ou incomplète.

4.2 Export SVG

Nous avons également ajouté un export au format SVG. Contrairement au PNG, le SVG est un format vectoriel : l'image n'est pas enregistrée comme un ensemble de pixels, mais comme une suite d'objets géométriques décrits textuellement.

Nous avons donc généré le fichier SVG à partir des données du graphe elles-mêmes. Les nœuds sont reconstruits comme des formes géométriques, et les arêtes comme des segments ou des courbes selon le mode d'affichage choisi. L'intérêt principal est qu'une telle image peut être agrandie sans perte de qualité, ce qui est utile pour une insertion dans un rapport.

4.3 Extensions liées à l'interface utilisateur

Nous avons ajouté plusieurs extensions côté interface afin d'améliorer le confort d'utilisation de l'application. La fenêtre JavaFX peut désormais être redimensionnée librement, ce qui permet d'adapter l'espace de travail à la taille de l'écran ou au type d'analyse souhaité.

Nous avons aussi introduit la possibilité d'annuler certaines actions effectuées par l'utilisateur. Cette fonctionnalité a été relativement simple à intégrer grâce au bus de commandes mis en place dans l'architecture Java. Ce mécanisme centralise les actions envoyées au moteur et permet d'en mémoriser l'historique, ce qui rend possible une logique d'undo/redo sans disperser ce comportement dans tous les contrôleurs.

4.4 Prévisualisation : courbes de Bézier et épaisseur des arêtes

Le panneau de prévisualisation a également été enrichi. Nous avons ajouté la possibilité d'afficher les arêtes sous forme de courbes de Bézier et de modifier leur épaisseur.

Dans le cas d'une arête droite, l'affichage est simple : il suffit de relier deux nœuds par un segment. Pour obtenir une arête courbe, il faut construire une géométrie différente. Nous avons utilisé pour cela des courbes de Bézier, définies à partir des deux extrémités de l'arête et d'un point de contrôle. Ce point de contrôle permet de régler la courbure et de mieux séparer visuellement certaines connexions lorsque le graphe est dense.

5

Critiques et extensions possibles

5.1 Évolutions possibles côté C

Une première évolution concerne la structure générale du code natif. La partie C repose encore sur un certain nombre de variables globales. Ce choix existait déjà dans la version de départ. Une évolution naturelle serait de regrouper progressivement l'état du moteur dans des structures mieux isolées, afin de faciliter les tests, les rechargements et les futures optimisations.

Le code C conserve aussi des tableaux statiques dimensionnés par des constantes maximales. Une amélioration naturelle serait de remplacer progressivement ces tableaux par des allocations dynamiques dimensionnées selon le graphe réellement chargé.

Un autre point qui pourrait être rediscuté concerne l'usage systématique du type `double`. Dans plusieurs calculs de rendu ou de simulation, une précision aussi élevée n'est pas toujours indispensable. L'utilisation de `float` réduirait l'empreinte mémoire et pourrait améliorer certains transferts de données, notamment lorsqu'il s'agit de manipuler un grand nombre de positions ou de valeurs intermédiaires. Ce choix demanderait cependant une vérification globale du moteur, car il ne suffit pas de remplacer les types localement.

Enfin, la question du parallélisme peut être rendue plus configurable. Le nombre de threads utilisés par certaines parties du moteur dépend encore largement de choix internes. Une évolution utile serait de rendre ce paramètre accessible, afin d'adapter plus finement le niveau de parallélisme à la machine de l'utilisateur.

5.2 Évolutions possibles côté rendu

Le rendu interactif conserve des compromis. La version finale utilise JOGL avec un `GLJPanel` intégré dans JavaFX via un `SwingNode`. Cette architecture permet de conserver le rendu OpenGL existant dans une interface JavaFX refactorisée.

Sur de grands graphes, afficher tous les sommets et toutes les arêtes en permanence reste coûteux, même avec un GPU. Les optimisations côté rendu réduisent une partie du coût, et les flame graphs permettent de voir précisément l'effet de ces optimisations.

Une piste future serait de mettre en place un LOD plus avancé, conçu dès le départ pour préserver la lisibilité du graphe. Un LOD bien intégré pourrait réduire la quantité de géométrie affichée lorsque l'utilisateur est très éloigné du graphe, puis réintroduire les détails progressivement lors du zoom.

La migration vers OpenGLFX/LWJGL reste également une piste possible. Les essais réalisés indiquent un gain potentiel mais elle implique de reprendre le cycle de vie du rendu, les interactions utilisateur, les buffers GPU et les exports.

5.3 Évolutions possibles du calcul des forces

Le calcul des forces reste le principal chantier d'optimisation. La version exacte par clusters réduit le nombre de comparaisons par rapport à une répulsion globale, mais elle reste sensible à la taille des clusters. La version par représentants améliore le coût sur les grands graphes et montre une piste intéressante pour la suite.

Dans la version dans le main, le k-means était parallélisé mais restait limité par un verrou global placé dans la boucle d'accumulation des centres. La branche `test_optim` tente de corriger ce point en utilisant des accumulateurs locaux par thread, puis une phase de réduction après synchronisation ce qui est une bonne piste pour réduire l'overhead de ce dernier.

La difficulté vient du compromis entre vitesse et qualité visuelle. Une répulsion plus précise donne souvent une meilleure disposition, mais elle devient coûteuse sur les grands graphes. À l'inverse, une approximation plus agressive accélère le calcul, mais elle doit être réglée pour conserver une bonne séparation des communautés.

Le k-means améliore le résultat dans les deux approches. Il aide à structurer les groupes et à réduire certains défauts visuels. La qualité finale dépend aussi de la manière dont les centres sont choisis, de la fréquence de mise à jour des groupes et de l'équilibre entre attraction et répulsion.

5.4 Extensions encore envisageables

Même si plusieurs fonctionnalités ont été ajoutées, certaines extensions utiles pourraient encore enrichir l'application. Il resterait pertinent d'ajouter des exports de données dans des formats comme CSV ou DOT, afin de faciliter la réutilisation du graphe dans d'autres outils.

D'autres améliorations concernent directement l'affichage. Il serait par exemple intéressant d'ajouter l'affichage optionnel des labels de nœuds et d'arêtes, ou encore des paramètres visuels supplémentaires inspirés d'outils comme Gephi, par exemple des contours plus marqués, davantage d'options de stylisation ou un contrôle plus fin de la prévisualisation.

Une autre évolution possible concerne l'anti-aliasing. À l'heure actuelle, le rendu interactif, contrairement à l'export, présente parfois un aliasing visible, en particulier sur les arêtes. L'ajout d'un anti-aliasing permettrait d'améliorer la qualité visuelle. Il serait préférable de proposer cette option sous la forme d'un paramètre activable par l'utilisateur.

Plus généralement, cette version finale constitue une base de travail plus claire que la version initiale. Les choix réalisés pendant le projet ont visé à stabiliser l'existant, à mieux structurer l'application et à rendre possibles de futures évolutions sans repartir de zéro.

6

Conclusion

Le projet PSTL nous a confrontés à une situation réaliste : reprendre un projet déjà avancé, mais difficile à lancer, à comprendre et à modifier. Le travail n'a donc pas consisté uniquement à ajouter des fonctionnalités. Il a d'abord fallu analyser l'existant, identifier les points bloquants, réorganiser le code et stabiliser les usages principaux.

La version finale améliore plusieurs aspects de la base initiale : l'architecture Java est plus lisible, l'interface est plus claire, le rendu OpenGL avec JOGL est mieux organisé, le chargement natif est mieux isolé, et certaines fonctionnalités comme l'export PNG/SVG ou la prévisualisation ont été ajoutées.

Le profilage avec **async-profiler** nous a aussi aidés à mieux comprendre les limites de l'application. Les flame graphs ont montré que les ralentissements venaient à la fois du calcul des forces, du rendu et des transferts entre les différentes couches. Ils ont surtout permis de mesurer l'effet des optimisations côté rendu et de mieux cibler les prochaines pistes de travail.

Nous avons aussi exploré des optimisations plus ambitieuses sur une branche séparée, **test_optim**. Ces essais ont porté sur le calcul des forces, la GUI, certaines optimisations du rendu et une tentative de migration vers OpenGLFX/LWJGL. Ils ont montré qu'il était possible d'aller plus loin sur de grands graphes, notamment en réduisant le coût du rendu et en explorant une répulsion par représentants.

Le projet nous a permis de progresser sur plusieurs aspects d'ingénierie logicielle : lecture d'un code existant, refactorisation, intégration native, rendu GPU et diagnostic de performances.

Annexes

.1 Version initiale

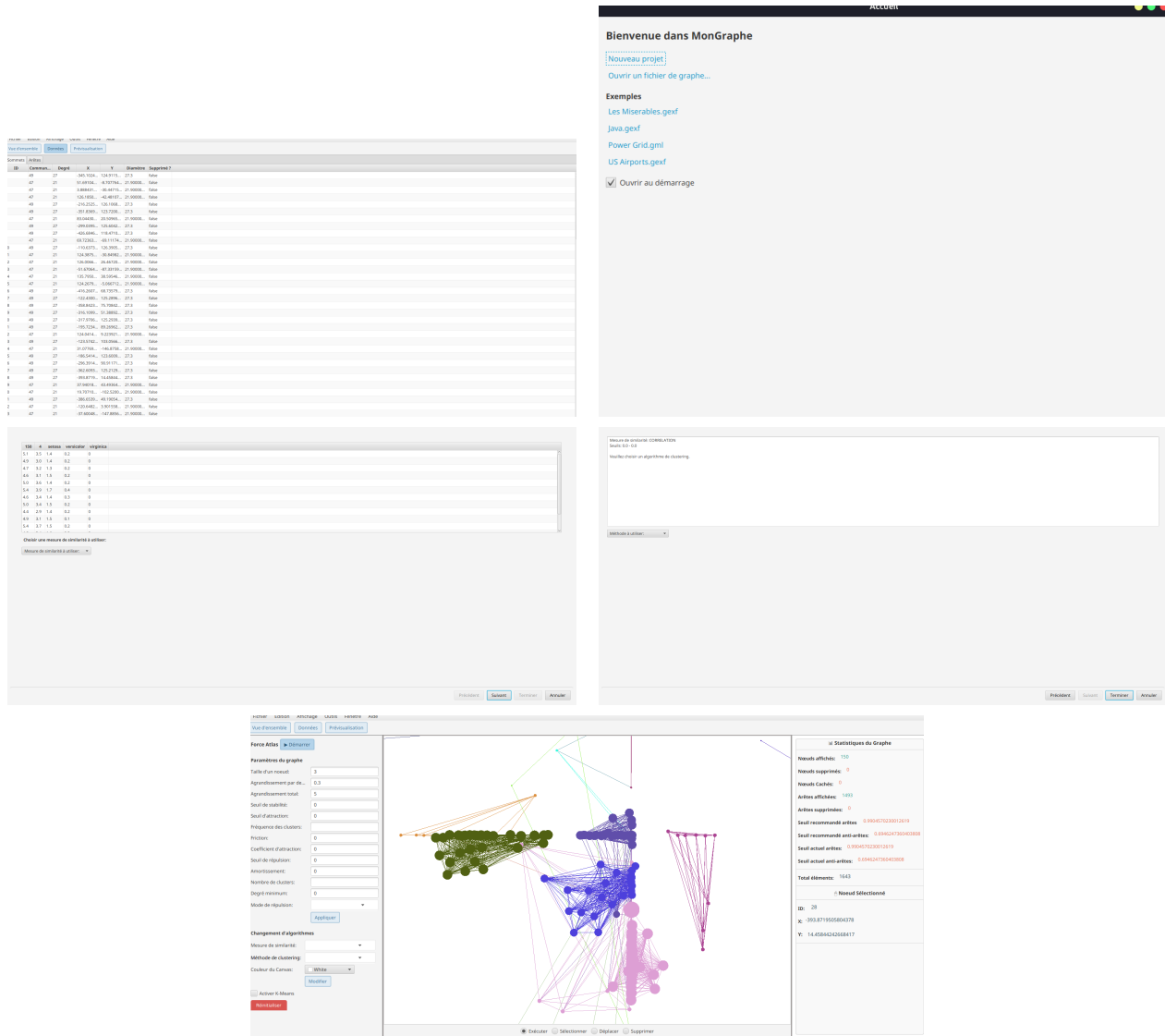


FIGURE 1 – Interface de la version initiale du projet.

.2 Version finale

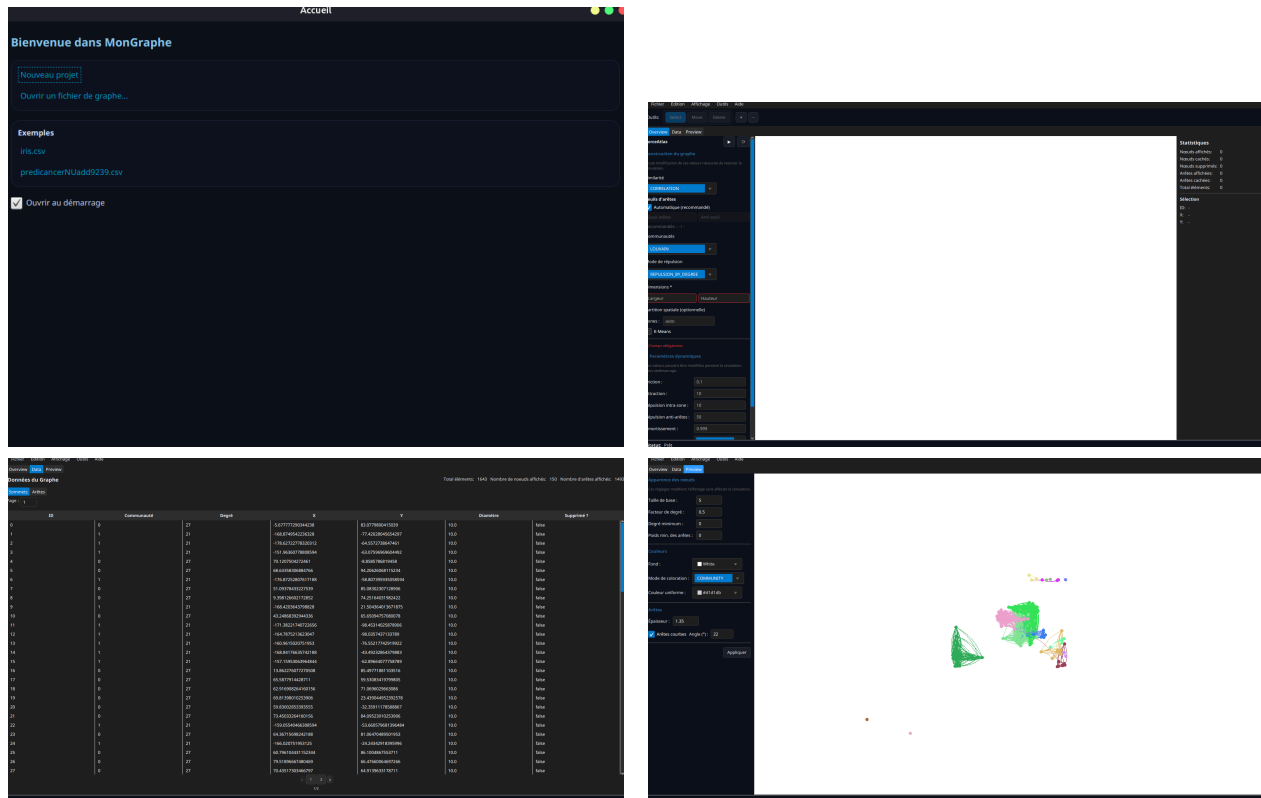


FIGURE 2 – Interface de la version finale retenue.

Bibliographie

- [1] Gephi. *Gephi - The Open Graph Viz Platform*. Site officiel. <https://gephi.org/>. Consulté le 16 avril 2026.
- [2] Mathieu Jacomy, Tommaso Venturini, Sebastien Heymann, Mathieu Bastian. *ForceAtlas2, a Continuous Graph Layout Algorithm for Handy Network Visualization Designed for the Gephi Software*. PLOS ONE, 2014. <https://doi.org/10.1371/journal.pone.0098679>.
- [3] OpenJFX. *Getting Started with JavaFX*. Documentation officielle. <https://openjfx.io/openjfx-docs/>. Consulté le 16 avril 2026.
- [4] OpenJFX. *JavaFX 21 Javadoc*. <https://openjfx.io/javadoc/21/index.html>. Consulté le 16 avril 2026.
- [5] OpenJFX. *Introduction to FXML*. https://openjfx.io/javadoc/25/javafx.fxml/javafx/fxml/doc-files/introduction_to_fxml.html. Consulté le 16 avril 2026.
- [6] JogAmp. *JOGL Userguide*. <https://jogamp.org/jogl/doc/userguide/>. Consulté le 16 avril 2026.
- [7] HuskerDev. *OpenGLFX*. <https://github.com/husker-dev/openglfx>. Consulté le 16 avril 2026.
- [8] LWJGL. *Lightweight Java Game Library*. <https://www.lwjgl.org/>. Consulté le 16 avril 2026.
- [9] Oracle. *Java Native Interface Specification*. <https://docs.oracle.com/en/java/javase/25/docs/specs/jni/index.html>. Consulté le 16 avril 2026.
- [10] Khronos Group. *OpenGL Registry*. https://registry.khronos.org/OpenGL/index_gl.php. Consulté le 16 avril 2026.
- [11] Khronos Group. *The OpenGL Shading Language, Version 4.60*. <https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.4.60.html>. Consulté le 16 avril 2026.
- [12] Graphviz. *DOT Language*. <https://graphviz.org/doc/info/lang.html>. Consulté le 16 avril 2026.
- [13] W3C. *Scalable Vector Graphics (SVG) 2*. <https://www.w3.org/TR/SVG2/>. Consulté le 16 avril 2026.
- [14] async-profiler. *Async-profiler*. Dépôt officiel. <https://github.com/async-profiler/async-profiler>. Consulté le 2 mai 2026.
- [15] Brendan Gregg. *Flame Graphs*. <https://www.brendangregg.com/flamegraphs.html>. Consulté le 2 mai 2026.